# D4 Data errors

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

## Contents

**Track D: Finding errors in code ..... 2**
D4 Data errors ..... 3
D4-L1 Row counts across layers ..... 3
D4-L2 Distribution drift between waves ..... 7
D4-L3 Weight totals and survey checks ..... 11
Track D best-practice checklist ..... 15
**Appendix: solutions and answer keys ..... 16**

# Track D: Finding errors in code

Find and fix errors fast, including errors in code that Claude and Copilot write.

# D4 Data errors

Roadmap: Every month. Row counts across layers, distribution drift and weight totals.

## D4-L1 Row counts across layers

### Concept

Every layer of a medallion pipeline changes the row count for a reason. Write the reasons down as rules, then check them on every run. A count that breaks a rule is a bug, even when no step failed.

For one tracker and one wave:

| Check | Rule |
|---|---|
| Bronze vs delivery | Bronze rows for the delivery = rows in the delivered file or manifest |
| Silver vs bronze | Silver rows = distinct respondent keys in bronze. Never more than bronze rows |
| Gold vs silver | Sum of gold `unweighted_n` = silver rows in scope |
| Redelivery | After MERGE, silver rows = rows before + rows inserted |

**Where the numbers come from.** Count each layer with one `agg` pass, not repeated `count()` calls on an uncached DataFrame: `df.agg(F.count("*"), F.count_distinct("respondent_id"))`. For writes, Delta records metrics in the table history. `DESCRIBE HISTORY` shows `numOutputRows` for a write, and for a MERGE `numSourceRows`, `numTargetRowsInserted` and `numTargetRowsUpdated`. Checked on delta-spark 4.0.1.

**Filter both sides the same way.** Your reference `get_unprocessed_waves` compares bronze waves **for one tracker** with silver waves **for all trackers**. When another tracker already has wave W14 in silver, this tracker's W14 is never loaded. Every reconciliation query filters bronze, silver and gold on the same tracker and wave.

**Store the result.** Write each reconciliation as a structured row in the run log: layer, expected, actual, status. A failed CRITICAL rule halts the pipeline before gold is published.

Counts catch lost and duplicated rows. They do not catch wrong values in the right number of rows. Lessons D4-L2 and D4-L3 cover those.

## Why it matters

Wave W14 for one tracker never reached silver. The job logged success every night for three weeks, because `get_unprocessed_waves` found W14 in silver, loaded by a different tracker. A rule "silver rows for tracker and wave > 0 when bronze rows > 0" would have failed on the first night.

## Worked example: weak code

```
1 bronze_n = bronze.count()
2 silver_n = silver.count()
2 if silver_n > 0:
4     print("silver loaded")
```

1. **Line 1.** All trackers and all waves. The number means nothing for this load.
2. **Line 3.** Any row from any tracker passes this check.

## Worked example: strong code

```
1  from pyspark.sql import functions as F
2
3  def layer_counts(df, tracker_id: str, wave_id: str) -> dict[str, int]:
4      """Rows and distinct respondents for one tracker and wave, in one
       pass."""
5      row = (
1          df.filter((F.col("tracker_id") == tracker_id) & (F.col("wave_id")
       == wave_id))
2          .agg(F.count("*").alias("rows"),
       F.count_distinct("respondent_id").alias("keys"))
8          .first()
9      )
10     return {"rows": row["rows"], "keys": row["keys"]}
11
12 bronze_c = layer_counts(bronze, "NAT", "W14")
13 silver_c = layer_counts(silver, "NAT", "W14")
3  if silver_c["rows"] != bronze_c["keys"]:
15     raise RuntimeError(f"silver {silver_c['rows']} != bronze keys
       {bronze_c['keys']}")
```

1. **Line 6.** The same tracker and wave filter on every layer.
2. **Line 7.** Both counts in one pass over the data.
3. **Line 14.** A written rule, checked every run. It halts before gold.

Write the expected relationship between layers as rules, compute counts in one pass on identical filters, and halt when a CRITICAL rule fails.

## Recap card

- **Row-count rules: silver vs bronze** Silver rows = distinct respondent keys in bronze for the tracker and wave.

- **Rows after a MERGE** Rows before + numTargetRowsInserted (minus any deletes). Updates add nothing.
- **Counting efficiently** One agg pass: F.count('*') and F.count_distinct(key). Not repeated count() calls.
- **The get_unprocessed_waves bug** Bronze filtered on tracker, silver not. Filter both sides on the same keys.

## Practice

### Exercise 1. Multiple choice (level 1)

Bronze holds 10,400 rows for tracker NAT, wave W14, with 10,000 distinct respondents. Silver deduplicates on respondent and wave. How many silver rows do you expect?

- a) 10,000
- b) 10,400
- c) Any number up to 10,400
- d) It depends on the weights

**Why?**

- r1) Silver's grain is one row per respondent per wave, so it holds one row per distinct key.
- r2) Silver always drops 4% of rows.

### Exercise 2. Multiple choice (level 2)

A redelivery MERGE into silver reports `numSourceRows = 500`, `numTargetRowsInserted = 120`, `numTargetRowsUpdated = 380`. Silver had 10,000 rows before. How many rows does it hold now?

- a) 10,120
- b) 10,500
- c) 10,380
- d) 10,000

**Why?**

- r1) Updates replace existing rows. Only inserts add rows.
- r2) MERGE never removes rows, so every source row is added.

### Exercise 3. Fill the gap (level 2)

Compute rows and distinct respondents in one pass.

```
1 row = df.agg(
```

```
    F.count("*").alias("rows"),
    F._____(1)("respondent_id").alias("keys"),
).____(2)()
```

### Exercise 4. Write the function (level 3)

Write `reconcile(delivered: int, bronze_rows: int, bronze_keys: int, silver_rows: int, gold_n: int, in_scope: int) -> list[dict[str, object]]`. Return one record per rule, in this order, each `{"rule": name, "expected": int, "actual": int, "status": "PASS" or "FAIL"}`: `"bronze_vs_delivery"` (bronze_rows equals delivered), `"silver_vs_bronze_keys"` (silver_rows equals bronze_keys), `"gold_vs_silver_scope"` (gold_n equals in_scope). Raise `ValueError` if any count is negative, or if `in_scope` exceeds `silver_rows`.

Your code must pass these checks:

- Three rules in order, all PASS for consistent counts.
- Silver must equal distinct bronze keys, not bronze rows. Record expected and actual.
- Gold unweighted_n must add up to the in-scope silver rows.
- Negative counts, or more in scope than silver rows, are impossible inputs. Raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Row-count rules across layers for one tracker and wave.
def reconcile(
    delivered: int,
    bronze_rows: int,
    bronze_keys: int,
    silver_rows: int,
    gold_n: int,
    in_scope: int,
) -> list[dict[str, object]]: ...
```

Starter code:

```
def reconcile(delivered, bronze_rows, bronze_keys, silver_rows, gold_n,
    in_scope):
    return [{"rule": "all", "status": "PASS" if silver_rows > 0 else
    "FAIL"}]
```

### Exercise 5. Spot the bug (level 4)

This is your reference `get_unprocessed_waves`, cut down. Tracker NAT's wave W14 is never loaded when another tracker already has W14 in silver. Tap the line with the bug, then fix it.

```
bronze_waves = (
```

```
    spark.table(bronze_table).filter(F.col("tracker_id") == tracker_id)
    .select("wave_id").distinct()
)
silver_waves = spark.table(silver_table).select("wave_id").distinct()
new_waves = [r.wave_id for r in
    bronze_waves.subtract(silver_waves).collect()]
```

Write down the line number of the bug. Then write the fix.

### Sources

- Delta MERGE history records numSourceRows, numTargetRowsInserted, numTargetRowsUpdated; writes record numOutputRows. (verified: Local run of DESCRIBE HISTORY on delta-spark 4.0.1 with PySpark 4.0.4, September 2026.)

## D4-L2 Distribution drift between waves

### Concept

Row counts can be perfect while the data inside has changed. A routing error drops one question. A supplier recodes region. A weighting run uses last year's targets. Each shows up as a **shift in distributions** between waves.

Three checks, from simple to sensitive:

- **Category shares.** For each categorical variable, compare each category's weighted share with the prior wave. Flag a move larger than a tolerance, say 5 percentage points. This is what your `check_wave_stability` does, once its truthiness bug is fixed (lesson D3-L3).
- **New and missing categories.** A category present in one wave and absent in the other is a recode until proven otherwise. `region = "Northern"` appearing beside `"North"` is a supplier change, not a population change.
- **Population stability index (PSI)** summarises a whole distribution in one number:

```
PSI = sum over categories of (actual - expected) x ln(actual / expected)
```

where `expected` and `actual` are shares in the prior and current wave. PSI is 0 when nothing moved. A category with a share of zero makes the logarithm undefined, so replace zero shares with a small floor, such as 0.0001, before computing. A common rule of thumb reads PSI below 0.1 as stable and above 0.25 as a large shift. Those cut-offs are convention, not a standard: calibrate them on your own history of waves.

**Guard the divisions.** Every share divides by a total. An empty wave, or a variable with no valid answers, gives a total of zero. Your reference `_check_distributions` divides by `total` with no guard and fails on an empty delivery. Check the total first and report "no data" as its own status.

Drift is a question, not a verdict. Real change happens. The check makes a person look before the numbers are published.

## Why it matters

A supplier renamed region code `"North"` to `"Northern"` in wave W21. Silver loaded both values, the region lookup matched only `"North"`, and the northern subgroup fell from 1,900 respondents to 12. Every row count passed. A new-and-missing category check on `region` would have flagged `"Northern"` on the first run.

## Worked example: weak code

```
  1 def share_changes(prior: dict[str, int], current: dict[str, int]) ->
      dict[str, float]:
  2     """Change in share per category."""
  3     p_total = sum(prior.values())
(1)   c_total = sum(current.values())
(2)   return {k: current[k] / c_total - prior[k] / p_total for k in prior}
```

1. **Line 4.** An empty wave makes this 0, and the division fails.
2. **Line 5.** Loops over prior only. A new category in current is ignored, and a missing one raises KeyError.

## Worked example: strong code

```
  1 def share_changes(prior: dict[str, int], current: dict[str, int]) ->
      dict[str, float]:
  2     """Change in share per category, over the union of categories.
  3
  4     Raises:
  5         ValueError: If either wave has no observations.
  6     """
  7     p_total = sum(prior.values())
  8     c_total = sum(current.values())
(1)   if p_total == 0 or c_total == 0:
 10         raise ValueError("a wave has no observations")
(2)   categories = sorted(prior.keys() | current.keys())
 12     return {
 13         k: current.get(k, 0) / c_total - prior.get(k, 0) / p_total for k
      in categories
 14     }
```

1. **Line 9.** An empty wave is reported, not divided by.

2. **Line 11.** New and missing categories both appear, with a share of 0 on the side where they are absent.

Compare shares over the union of categories, flag new and missing values, summarise with PSI, and guard every total.

## Recap card

- **Three drift checks** Category shares vs prior wave, new and missing categories, and a PSI summary.
- **PSI formula** Sum of (actual - expected) x ln(actual / expected) over shares, with zero shares floored.
- **PSI thresholds 0.1 and 0.25** A rule of thumb, not a standard. Calibrate on your own wave history.
- **Percentage points vs relative change** 30% to 45% is +15 points and +50% relative. Say which a tolerance uses.

## Practice

### Exercise 1. Multiple choice (level 1)

Wave W20 has region values North, South, East. Wave W21 has Northern, South, East. Row counts match. What is the most likely cause?

- a) A recode by the supplier
- b) The population of the North moved
- c) A weighting error
- d) Normal sampling variation

**Why?**

- r1) A category that vanishes while a similar new one appears is a label change until proven otherwise.
- r2) Suppliers always rename categories between waves.

### Exercise 2. Predict the output (level 2)

What does this print?

```
prior = {"yes": 30, "no": 70}
current = {"yes": 45, "no": 55}
change = current["yes"] / 100 - prior["yes"] / 100
print(round(change * 100))
```

### Exercise 3. Fill the gap (level 2)

Complete the PSI term for one category, with zero shares floored.

```python
import math

e = max(expected_share, FLOOR)
a = max(actual_share, FLOOR)
term = (a - e) * math._____(1)(a / _____(2))
```

### Exercise 4. Write the function (level 3)

Write `psi(expected: dict[str, int], actual: dict[str, int], floor: float = 0.0001) -> float`. Both map categories to counts. Use the union of categories, a missing category counting 0. Convert counts to shares, replace shares below `floor` with `floor`, and return the sum of `(a - e) * ln(a / e)`. Raise `ValueError` if either total is 0.

Your code must pass these checks:

- Identical shares give PSI 0, whatever the totals.
- Use shares, not counts, and the natural logarithm.
- Include categories from both waves. Floor a zero share at 0.0001 instead of dividing by zero.
- An empty wave has no shares. Raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Population stability index between two categorical distributions.
def psi(
    expected: dict[str, int],
    actual: dict[str, int],
    floor: float = 0.0001,
) -> float: ...
```

Starter code:

```python
import math


def psi(expected, actual, floor=0.0001):
    return sum((actual[k] - expected[k]) * math.log(actual[k] / expected[k]) for k in expected)
```

### Exercise 5. Spot the bug (level 3)

This is your reference `_check_distributions` logic, as a plain function. It fails on an empty delivery. Tap the line, then fix it: return `"NO_DATA"` when the total is 0.

```python
def distribution_status(value_count: int, total: int, min_pct: float, max_pct: float) -> str:
    """PASS if the value's share is within bounds, else WARN."""
    pct = value_count / total
```

```
    return "PASS" if min_pct <= pct <= max_pct else "WARN"
```

Write down the line number of the bug. Then write the fix.

### Sources

- PSI = sum (a - e) ln(a / e) over category shares; zero shares need a floor. (verified: Definition checked by the exercise tests in this repo; the formula follows from its terms.)
- PSI thresholds of 0.1 and 0.25 are an informal industry convention. (unverified: Widely quoted in credit-scoring practice; no official standard located. Taught as a rule of thumb only.)

## D4-L3 Weight totals and survey checks

### Concept

Survey weights carry the whole estimate. A wrong weight gives a plausible but wrong percentage in every table. Check them on every wave, before gold.

**Totals and range:**

- No null weights, and none zero or negative, unless your design allows zero.
- The weight sum matches its target: the population total for grossing weights, or the sample size for weights normalised to mean 1. Your contract's `weight_sum_min` and `weight_sum_max` encode this.
- The ratio of largest to smallest weight stays below an agreed limit. One respondent standing for 40 others dominates their subgroup.

**Efficiency.** Unequal weights cost precision. Kish's approximation measures it:

```
deff  = n x sum(w^2) / (sum w)^2 = 1 + CV^2
n_eff = (sum w)^2 / sum(w^2) = n / deff
```

Here CV is the population coefficient of variation of the weights. Weights 0.5, 1.0, 1.5 and 3.0 give a deff of 1.39 and an effective sample of 2.88 out of 4. Track n_eff per subgroup and wave. A sudden fall means the weighting struggled, often because a quota cell was nearly empty.

**Spark details that break these checks.** Checked on PySpark 4.0.4:

- `F.stddev` is the **sample** standard deviation. It returns null for a single row, and the Kish identity uses the population version, `F.stddev_pop`.

- On an empty DataFrame, `mean` and `stddev` return null. Your reference `check_weight_distribution` then calls `round(None, 4)` and raises `TypeError`.
- `F.sum(F.when(F.col("reach_flag") == 1, 1))` is **null**, not 0, for a subgroup where nobody was reached. Your gold table then shows a blank instead of 0% reach. Wrap it in `F.coalesce(..., F.lit(0))`.

Decide what each result means for the run. A null weight is CRITICAL: halt. A deff above your usual range is WARN: log it and have someone look.

### Why it matters

A weighting run used last year's population targets for one age band. The weight sum stayed within tolerance overall, so the contract check passed. The 55+ band's effective sample fell from 800 to 190, and its reach estimate jumped 9 points. A per-subgroup n_eff check with a WARN threshold would have flagged it before the dashboard refreshed.

### Worked example: weak code

```
1 stats = df.agg(F.mean(w).alias("mean"), F.stddev(w).alias("sd")).first()
2 cv = stats["sd"] / stats["mean"] if stats["mean"] else None
2 result = {"mean": round(stats["mean"], 4), "cv": round(cv, 4) if cv else
    None}
```

1. **Line 1.** Sample standard deviation, and null for a single row.
2. **Line 3.** round(None, 4) raises TypeError on an empty wave. `if cv` treats a CV of 0 as missing.

### Worked example: strong code

```
1 row = df.agg(
2     F.count(w).alias("n"),
3     F.sum(w).alias("total"),
1     F.sum(F.col(w) * F.col(w)).alias("total_sq"),
5 ).first()
2 if row["n"] == 0 or row["total"] is None or row["total"] == 0:
7     raise ValueError("no weights to check")
8 deff = row["n"] * row["total_sq"] / row["total"] ** 2
9 n_eff = row["total"] ** 2 / row["total_sq"]
```

1. **Line 4.** Kish needs only n, sum and sum of squares, in one pass.
2. **Line 6.** Empty and zero totals are reported, never divided by.

Check nulls, range, sum and efficiency per wave and subgroup. Mind Spark's nulls: sample stddev, empty aggregates and sums over no matches.

## Recap card

- **Kish design effect and effective sample** deff = n x sum(w^2) / (sum w)^2 = 1 + CV^2. n_eff = (sum w)^2 / sum(w^2).
- **F.stddev in Spark** Sample standard deviation, null for one row. Use F.stddev_pop for 1 + CV^2.
- **sum(when(cond, 1)) with no matches** Null, not 0. Wrap in coalesce(..., lit(0)) or add otherwise(0).
- **Severity for a null weight** CRITICAL: halt before gold.

## Practice

### Exercise 1. Multiple choice (level 1)

In gold, `F.sum(F.when(F.col("reach_flag") == 1, 1))` gives what for a subgroup where no one was reached?

- a) NULL
- b) 0
- c) An error
- d) The subgroup is dropped

**Why?**

- r1) when without otherwise gives null for non-matching rows, and the sum of only nulls is null.
- r2) Spark sums return null for any group with a zero.

### Exercise 2. Predict the output (level 2)

What does this print?

```
w = [1.0, 1.0, 1.0, 1.0]
n = len(w)
deff = n * sum(x * x for x in w) / sum(w) ** 2
print(deff)
```

### Exercise 3. Fill the gap (level 2)

Make the unweighted reach count 0 when no one in the subgroup was reached.

```
F._____(1)(F.sum(F.when(F.col("reach_flag") == 1, 1)),
    F.lit(_____(2))).alias("unweighted_reach")
```

### Exercise 4. Write the function (level 3)

Write

`weight_report(weights: list[float | None], max_ratio: float = 20.0) ->`

`dict[str, object]`. Return `{"status": ..., "n": ..., "n_eff": ..., "deff": ...}`. Status is `"CRITICAL"` if the list is empty or any weight is `None` or not positive (and then `n_eff` and `deff` are `None`), `"WARN"` if max/min weight exceeds `max_ratio`, otherwise `"PASS"`. Use `n_eff = (sum w)^2 / sum(w^2)` and `deff = n / n_eff`, each rounded to 4 decimals.

Your code must pass these checks:

- Weights 0.5, 1.0, 1.5, 3.0 give n_eff 2.88 and deff 1.3889.
- Empty lists, None, zero and negative weights are CRITICAL, with no efficiency figures.
- A max/min ratio of 30 exceeds 20: WARN.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Weight checks with Kish's effective sample size and design effect.
def weight_report(
    weights: list[float | None],
    max_ratio: float = 20.0,
) -> dict[str, object]: ...
```

Starter code:

```python
def weight_report(weights, max_ratio=20.0):
    n_eff = sum(weights) ** 2 / sum(w * w for w in weights)
    return {"status": "PASS", "n": len(weights), "n_eff": n_eff, "deff":
    len(weights) / n_eff}
```

### Exercise 5. Architecture design (level 4)

Set severities for weight checks in the silver QA step.

Options: CRITICAL: halt before gold, WARN: drop those rows and continue.

1. Any null final_weight
2. Subgroup deff 40% above its usual range
3. Standard deviation for the CV

## Sources

- Kish deff = n sum(w^2)/(sum w)^2 = 1 + CV^2 (population CV); n_eff = (sum w)^2 / sum(w^2); example weights give 1.3889 and 2.88. (verified: Computed locally on CPython 3.13, September 2026; identity follows algebraically.)
- PySpark 4.0.4: sum(when(...)) over no matches is null; stddev is the sample version and null for one row; mean and stddev of an empty DataFrame are null. (verified: Local run on PySpark 4.0.4, September 2026.)

## Track D best-practice checklist

☐ Read a traceback from the bottom up. Find the first frame in your own code.

☐ Reproduce the error on the smallest data that shows it.

☐ Check row counts before and after every join.

☐ Test numbers with `is None`, never with truthiness, when zero is a valid value.

☐ Never use a mutable object as a default argument.

☐ Check every function and parameter an AI suggests against the official docs for your version.

☐ Run AI code on a small input with a known answer before you trust it.

☐ Write the failing test first. A test that only repeats the code proves nothing.

☐ Ask the model to list its assumptions. Check each one.

☐ State versions, constraints, expected outputs and tests in every prompt.

# Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

## D4-L1 Row counts across layers

### D4-L1 exercise 1

**Hints.** (1) What is silver's grain? (2) One row per respondent per wave. (3) Count distinct keys in bronze.

**Answer.** a) 10,000 Because: r1) Silver's grain is one row per respondent per wave, so it holds one row per distinct key.

The grain turns a vague expectation into an exact rule: silver rows = distinct bronze keys.

**Common wrong answers.**

- `b`: That keeps the 400 duplicates. Silver deduplicates on the key.
- `c`: The grain fixes the number exactly. A vague rule catches nothing.
- `d`: Weights do not change row counts.
- `a|r2`: Right answer, wrong reason. The number follows the grain, not a fixed rate.

### D4-L1 exercise 2

**Hints.** (1) Which metric adds rows? (2) Inserts. (3) 10,000 + 120.

**Answer.** a) 10,120 Because: r1) Updates replace existing rows. Only inserts add rows.

Check after every MERGE: rows after = rows before + numTargetRowsInserted - rows deleted.

**Common wrong answers.**

- `b`: 380 source rows updated existing rows, so they add nothing.
- `c`: Updated rows were already counted in the 10,000.
- `d`: 120 new respondents were inserted.
- `a|r2`: Right answer, wrong reason. Updated rows replace existing ones and add nothing.

### D4-L1 exercise 3

**Hints.** (1) Which function counts distinct values? (2) Then fetch the single result row. (3) `count_distinct` and `first`.

**Answer.**

```python
row = df.agg(
    F.count("*").alias("rows"),
    F.count_distinct("respondent_id").alias("keys"),
).first()
```

One `agg` returns both numbers in one scan. `first()` brings the single row back to the driver.

**Common wrong answers.**

- g1 = `count`: count counts non-null values, duplicates included. You need distinct keys.

### D4-L1 exercise 4

**Hints.** (1) Validate inputs first. (2) List the rules as (name, expected, actual) tuples. (3) Silver is compared with distinct bronze keys.

**Answer.**

```python
def reconcile(
    delivered: int,
    bronze_rows: int,
    bronze_keys: int,
    silver_rows: int,
    gold_n: int,
    in_scope: int,
) -> list[dict[str, object]]:
    """Row-count rules across layers for one tracker and wave.

    Args:
        delivered: Rows in the delivery manifest.
        bronze_rows: Bronze rows for the delivery.
        bronze_keys: Distinct respondent keys in bronze.
        silver_rows: Silver rows for the wave.
        gold_n: Sum of gold unweighted_n.
        in_scope: Silver rows in scope.

    Returns:
        One structured record per rule.

    Raises:
        ValueError: If inputs are impossible.
    """
    counts = (delivered, bronze_rows, bronze_keys, silver_rows, gold_n,
in_scope)
    if min(counts) < 0:
```

```
        raise ValueError("counts must not be negative")
    if in_scope > silver_rows:
        raise ValueError("in_scope cannot exceed silver_rows")
    rules = [
        ("bronze_vs_delivery", delivered, bronze_rows),
        ("silver_vs_bronze_keys", bronze_keys, silver_rows),
        ("gold_vs_silver_scope", in_scope, gold_n),
    ]
    return [
        {"rule": name, "expected": expected, "actual": actual,
         "status": "PASS" if expected == actual else "FAIL"}
        for name, expected, actual in rules
    ]
```

Structured records with expected and actual values go straight into the run log, and a dashboard can chart them.

### D4-L1 exercise 5

**Hints.** (1) Compare the filters on both sides. (2) Which side has no tracker filter? (3) Line 5.

**Answer.** The bug is on line 5.

Fixed version:

```
bronze_waves = (
    spark.table(bronze_table).filter(F.col("tracker_id") == tracker_id)
    .select("wave_id").distinct()
)
silver_waves = spark.table(silver_table).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
new_waves = [r.wave_id for r in
    bronze_waves.subtract(silver_waves).collect()]
```

Both sides of a comparison need the same keys. Silver waves for all trackers hide this tracker's missing waves.

**Common wrong answers.**

- Line 6: subtract is the right operation. Its right side holds every tracker's waves.
- Line 2: The bronze side is filtered correctly. The silver side is not.

## D4-L2 Distribution drift between waves

### D4-L2 exercise 1

**Hints.** (1) Compare the labels, not the counts. (2) North vanished, Northern appeared. (3) A recode.

**Answer.** a) A recode by the supplier Because: r1) A category that vanishes while a similar new one appears is a label change until proven otherwise.

Check new and missing categories before comparing shares.

**Common wrong answers.**

- `b`: A population does not rename itself. Check the labels first.
- `c`: Weights change shares, not category names.
- `d`: Sampling variation moves shares. It does not create new labels.
- `a|r2`: Right answer, wrong reason. Most waves have no recode, which is why a check must catch the one that does.

### D4-L2 exercise 2

**Hints.** (1) Both totals are 100. (2) 0.45 - 0.30. (3) In points, times 100.

**Answer.**

```
15
```

A move from 30% to 45% is 15 percentage points, and a 50% relative increase. State which one a tolerance means.

**Common wrong answers.**

- `50`: That is the relative change (45/30 - 1). The check uses percentage points: 45% - 30%.
- `0`: The shares differ by 0.15, which is 15 points.

### D4-L2 exercise 3

**Hints.** (1) Natural logarithm. (2) Of actual over expected. (3) `log` and `e`.

**Answer.**

```
import math

e = max(expected_share, FLOOR)
a = max(actual_share, FLOOR)
term = (a - e) * math.log(a / e)
```

`(a - e) * ln(a / e)` is never negative, so every category adds to the index.

**Common wrong answers.**

- g1 = `log10`: PSI uses the natural logarithm, math.log.

### D4-L2 exercise 4

**Hints.** (1) Totals first, and guard zero. (2) Loop over `expected.keys() | actual.keys()`. (3) Floor each share with `max(share, floor)`.

**Answer.**

```python
import math


def psi(expected: dict[str, int], actual: dict[str, int], floor: float = 0.0001) -> float:
    """Population stability index between two categorical distributions.

    Args:
        expected: Category counts in the reference wave.
        actual: Category counts in the current wave.
        floor: Minimum share, so a zero share does not break the logarithm.

    Returns:
        PSI, 0 when the shares are identical.

    Raises:
        ValueError: If either wave has no observations.
    """
    e_total = sum(expected.values())
    a_total = sum(actual.values())
    if e_total == 0 or a_total == 0:
        raise ValueError("a wave has no observations")
    index = 0.0
    for category in expected.keys() | actual.keys():
        e = max(expected.get(category, 0) / e_total, floor)
        a = max(actual.get(category, 0) / a_total, floor)
        index += (a - e) * math.log(a / e)
    return index
```

Shares, the union of categories and a floor. Without the union, the recode that matters most adds nothing to the index.

### D4-L2 exercise 5

**Hints.** (1) What is total for an empty delivery? (2) Zero. (3) Check it before dividing.

**Answer.** The bug is on line 3.

Fixed version:

```python
def distribution_status(value_count: int, total: int, min_pct: float, max_pct: float) -> str:
    """PASS if the value's share is within bounds, WARN if not, NO_DATA if empty."""
    if total == 0:
        return "NO_DATA"
    pct = value_count / total
    return "PASS" if min_pct <= pct <= max_pct else "WARN"
```

Guard every division. An empty delivery is a status to report, not a crash.

**Common wrong answers.**

- Line 4: The bounds check is fine. The division above it fails first.

## D4-L3 Weight totals and survey checks

### D4-L3 exercise 1

**Hints.** (1) What does when() return when no otherwise is given? (2) Null. (3) Sum of nulls.

**Answer.** a) NULL Because: r1) when without otherwise gives null for non-matching rows, and the sum of only nulls is null.

Add `.otherwise(0)` inside, or `F.coalesce(..., F.lit(0))` outside.

**Common wrong answers.**

- `b`: That is what you want, and what coalesce(..., 0) gives you. The raw sum is null.
- `c`: No error: the null passes silently into the table.
- `d`: groupBy keeps the subgroup. Its sum is null.
- `a|r2`: Right answer, wrong reason. Zeros sum normally. Only an all-null input gives null.

### D4-L3 exercise 2

**Hints.** (1) sum(w^2) = 4, (sum w)^2 = 16. (2) 4 x 4 / 16. (3) 1.0.

**Answer.**

```
1.0
```

Equal weights give deff 1 and n_eff = n. Any variation raises deff above 1.

**Common wrong answers.**

- `4.0`: 4 x 4 / 16 = 1. Equal weights cost no precision.
- `0.0`: deff is at least 1. Equal weights give exactly 1.

### D4-L3 exercise 3

**Hints.** (1) Which function returns the first non-null argument? (2) The replacement value. (3) `coalesce` and `0`.

**Answer.**

```
F.coalesce(F.sum(F.when(F.col("reach_flag") == 1, 1)),
    F.lit(0)).alias("unweighted_reach")
```

`coalesce` returns its first non-null argument, so a null sum becomes 0.

**Common wrong answers.**

- g1 = `isnull`: isnull tests for null and returns a boolean. coalesce replaces it.

### D4-L3 exercise 4

**Hints.** (1) Check emptiness and invalid weights first. (2) Test `w is not None and w > 0`, not truthiness. (3) n_eff = total**2 / sum of squares; deff = n / n_eff.

**Answer.**

```python
def weight_report(weights: list[float | None], max_ratio: float = 20.0) -> dict[str, object]:
    """Weight checks with Kish's effective sample size and design effect.

    Args:
        weights: One weight per respondent. None means missing.
        max_ratio: Largest allowed max/min weight ratio before WARN.

    Returns:
        status ("PASS", "WARN" or "CRITICAL"), n, n_eff and deff.
    """
    n = len(weights)
    valid = [w for w in weights if w is not None and w > 0]
    if n == 0 or len(valid) != n:
        return {"status": "CRITICAL", "n": n, "n_eff": None, "deff": None}
    total = sum(valid)
    n_eff = total**2 / sum(w * w for w in valid)
    status = "WARN" if max(valid) / min(valid) > max_ratio else "PASS"
    return {"status": status, "n": n, "n_eff": round(n_eff, 4), "deff": round(n / n_eff, 4)}
```

A missing or non-positive weight is a data error to report, never a row to drop quietly.

### D4-L3 exercise 5

**Hints.** (1) Which result makes the estimate undefined? (2) Which result needs judgement? (3) Which standard deviation does Kish use?

**Answer.**

- A null weight makes every estimate for that respondent undefined: halt.
- A high deff is a signal for a person, not proof of an error: warn.
- Kish's identity uses the population standard deviation. Guard the empty case.

CRITICAL for undefined estimates, WARN for signals, and the right statistic for the formula.

**Common wrong answers.**

- nulls = `warn`: Dropping rows changes the sample silently. Halt and fix the weighting.
- deff = `critical`: Real populations shift. Halting on every efficiency change blocks legitimate waves.
- sd = `samp`: F.stddev is the sample version, null for one row, and does not match 1 + CV^2.